

When Does Graph Expansion Help Personal-Corpus Retrieval? A Query-Class-Stratified Evaluation Protocol for Retrieval over Human-Curated Wikilink Graphs, with Pilot Telemetry from a Production Deployment

Anton Dzyatkovskii 

Platinum Software Development Company, Brisbane, Australia

dzyatkovskiy.a2@gmail.com

July 2026

Abstract

Graph-augmented retrieval (Graph RAG) systems conventionally construct their graph by LLM-based entity and relation extraction. We study an alternative setting, common in personal knowledge management yet under-evaluated: the graph already exists as *human-curated wikilinks* accumulated in a linked note corpus. We describe a deployed retrieval system over such a corpus (dense retrieval with multilingual-e5, a bounded 1-hop wikilink expansion used strictly for candidate generation, and cross-encoder reranking; SQLite persistence; no graph database and no extraction pass), and report pilot A/B telemetry from production use: on 10 logged production queries compared vector-only versus vector-plus-graph, the graph hop changed the reranked top-12 in 5 cases (promoting 1–3 notes each, all arriving via the wikilink walk) and changed nothing in the other 5. The pilot also surfaced a directional failure mode: expansion appears to help thematic queries and hurt named-entity queries, whose hub-like cards flood the candidate pool. These observations motivate the paper’s main contribution: a *query-class-stratified evaluation protocol* (entity, theme, bridge, compare, temporal strata) for measuring when graph expansion helps, together with ablations (hop depth, neighbour cap, entity gate) and a paired statistical analysis plan. The pilot sample is small ($N=10$ queries, one corpus, one user) and we state this plainly; the protocol, not the pilot, is the contribution. Code for the retrieval layer is public; the protocol is designed so that any owner of a linked corpus can replicate it on private data without disclosing that data.

1 Introduction

Retrieval-augmented generation over personal knowledge bases increasingly incorporates graph structure. The dominant recipe builds that structure automatically: an LLM extraction pass mines entities and relations from documents, the triples populate a graph store, and retrieval traverses the result. This recipe carries three costs: an extraction pass that can hallucinate edges, a graph database to operate, and an ETL pipeline that can drift out of sync with the corpus.

A large and growing population of corpora, however, already contains a graph that no model produced: users of linked note-taking systems (Obsidian, Logseq, Roam, plain-Markdown wikis) place `[[wikilinks]]` by hand, over years. Each such edge is a deliberate, human-curated assertion that two notes are related. Whether retrieval over such corpora benefits from traversing these edges

— and, more precisely, *for which classes of queries* — is, to our knowledge, not systematically evaluated in the Graph RAG literature, which concentrates on automatically extracted graphs.

This paper makes three contributions, in ascending order of importance:

1. **System description.** A minimal production retrieval stack over a human-curated wikilink corpus: dense retrieval for entry points, a bounded 1-hop wikilink expansion used *only* as candidate generation (never as a ranking signal), and a cross-encoder reranker as the sole arbiter. The graph is parsed from the notes at query time and never materialized in a database, eliminating graph-corpus drift by construction (§3).
2. **Pilot telemetry.** Logged A/B measurements from production use, with an honest sample size ($N=10$ paired queries), reported descriptively rather than inferentially (§4).
3. **Evaluation protocol.** A query-class-stratified benchmark design — entity, theme, bridge, compare, temporal — with paired per-query comparisons, ablations, and a pre-registered analysis plan, intended to answer not “does graph help RAG” but *for which query classes* it helps, harms, or is inert (§5).

We claim no novelty for the retrieval ingredients themselves; the vector-plus-graph-plus-rerank pattern is established. The claims of this paper are the human-curated-graph setting, the candidate-generation-only discipline, and the stratified protocol.

2 Related work and honest prior art

- **Automatically extracted graph memory.** Microsoft GraphRAG builds an LLM-extracted entity graph with community summaries for corpus-level question answering [1]. Graphiti/Zep maintain a temporally-aware knowledge graph combining semantic, lexical, and graph search [2]. HippoRAG runs Personalized PageRank over an automatically constructed KG for multi-hop retrieval [3]. These systems are the closest prior art for our retrieval topology; all construct the graph by model extraction rather than consuming a human-curated one.
- **Agentic and tiered memory.** A-MEM organizes agent memory in a Zettelkasten style in which the *agent* links notes [4]; MemGPT/Letta manage tiered memory with self-editing [5]. Neither evaluates retrieval over pre-existing human-placed links.
- **Retrieve-then-rerank.** Dense retrieval followed by cross-encoder reranking is standard practice [6]; we use it unmodified.
- **Stratified IR evaluation.** Query-class-conditioned analysis of retrieval effectiveness has long precedent in IR (e.g., per-topic-type analyses in TREC tracks) [7]. Our protocol applies this discipline to the specific question of graph expansion over personal corpora.

We therefore claim **no novelty** for (a) dense-plus-rerank retrieval, (b) graph-assisted candidate generation as a mechanism, or (c) stratified evaluation as a methodology. The contribution is their combination in the human-curated-wikilink setting, the explicit candidate-generation-only constraint, and a replicable protocol for the query-class question, together with pilot production telemetry.

3 System

3.1 Corpus

The deployment corpus is a single user’s personal knowledge base accumulated over approximately fifteen years: roughly 176,000 Markdown files in total, of which a curated slice of 4,591 files (concepts, decisions, insights, people and project cards) is indexed for semantic retrieval as 7,557 chunks. The remainder (raw chat archives, session logs, ledgers) is deliberately excluded from the semantic index and served by lexical search, following an essence/evidence split; contamination measurements motivating that split are reported in the deployment’s engineering notes and are out of scope here.

3.2 Retrieval pipeline

1. **Entry points.** Dense retrieval with multilingual-e5 embeddings over the curated slice returns the top vector hits.
2. **Candidate generation by graph expansion.** For each top hit, the system parses that note’s wikilinks *at query time* and adds 1-hop neighbours to the candidate pool, bounded at 40 neighbours per query. Edges receive no score and confer no bonus.
3. **Reranking.** A cross-encoder reranks the merged pool (vector hits plus graph-arrived candidates) jointly; the top 12 are returned.

Two design constraints define the system. First, *expansion is candidate generation, not ranking*: a graph-arrived note wins only if the reranker independently judges it relevant to the query, so the failure mode of a bad hop is “reranker ignores it” rather than “noise reaches the reader.” Second, *the graph is never materialized*: edges are re-parsed from note text on every query, so graph and corpus cannot drift apart; there is no graph database and no extraction pass. Persistence (embeddings, telemetry) is a single SQLite file.

3.3 Telemetry substrate

Every production query can be run as a logged A/B pair — vector-only versus vector-plus-graph — with the diff of the reranked top-12 written to an `ab_recall` table. A separate deterministic per-turn ledger (7,219 assistant turns over 48 days at zero LLM-token cost, implemented as transcript parsing in stdlib Python) provides usage context but plays no role in retrieval.

4 Pilot telemetry

We report the pilot descriptively. Between the feature’s deployment and the time of writing, 10 production queries were logged as strict A/B pairs.

- In **5 of 10** pairs, the graph hop changed the final top-12, promoting 1–3 notes per query. Every promoted note entered the pool via the wikilink walk (i.e., none was a rerank-order artifact).
- In **5 of 10** pairs, the output was identical. Under the candidate-generation-only design this is an expected profile rather than a failure: when the vector lane already saturates the answer, a well-behaved expansion should be inert.

- **Directional failure observation.** Thematic queries (e.g., a topic phrased as a concept) benefited; named-entity queries (a person, a tool) were harmed in inspection: an entity’s card functions as a hub node whose 1-hop neighbourhood is large and weakly query-relevant, flooding the candidate pool. The deployed mitigation is a conservative lexical heuristic (`looks_like_entity()`) that disables expansion for name-shaped queries.

These observations are directional only. $N=10$ paired queries on one corpus for one user supports no inferential claim; the pilot’s role is to motivate the stratification hypothesis — that the effect of graph expansion is conditioned on query class — and to demonstrate that the measurement substrate operates in production. The protocol of §5 is designed to test that hypothesis properly.

5 Query-class-stratified evaluation protocol

5.1 Strata

Queries are classified into five strata, chosen to separate mechanisms by which graph structure could plausibly help or harm:

Entity. The query names a specific person, tool, project, or organization. Hypothesized *harm* regime (hub-node flooding).

Theme. The query names a topic or concept without naming an entity. Hypothesized *help* regime (adjacent curated notes recovered).

Bridge. The answer resides in a note lexically and semantically dissimilar to the query but linked from a note that is similar (one association away). Hypothesized *strongest-help* regime; this is the case wikilinks exist to serve.

Compare. The query asks for a relation or contrast between two items. Tests whether expansion from two entry regions merges usefully.

Temporal. The query is anchored to a time or sequence (“what did I decide about X in June”). Tests interaction of graph expansion with recency and versioning metadata.

5.2 Query sourcing and labeling

Queries are drawn from the production query log (real information needs, not synthetic paraphrases), classified into strata by two independent annotators with adjudication; stratum classification disagreement is itself reported (construct validity of the strata). Relevance labels over the union of both systems’ outputs are graded (0–2) by the corpus owner — the only person able to judge relevance on a personal corpus — with the system condition blinded and output order randomized at labeling time.

Two properties of that log constrain this step, and we state them rather than design around them silently. First, the log is *per node*: the A/B table is excluded from cross-machine synchronisation, so each machine accumulates its own query history and none observes the fleet’s. A study must therefore either pool every reachable log or declare which node’s habits it sampled. Second, the logged queries are overwhelmingly long topical questions, so the **entity** stratum is nearly empty and the **bridge** stratum is absent: a bridge query is defined by where its answer lives and cannot be recognised from surface form. Filling those strata requires authored seed queries, which is an obvious avenue for flattering the graph. We therefore record provenance (**log** or **seed**) per query, report the

mix alongside every per-stratum result, and treat a seed-heavy stratum as weaker evidence than a log-heavy one.

5.3 Conditions and ablations

Paired conditions per query: (A) vector-only; (B) vector + 1-hop expansion. Ablations on (B): hop depth (1 vs. 2), neighbour cap (10/40/unbounded), entity gate (on/off), and reranker pool size, each varied independently.

5.4 Metrics and analysis plan

Primary metric: nDCG@12 paired difference (B – A) per stratum; secondary: recall@12 of labeled relevant notes, promotion rate (fraction of queries where B changes top-12), and *harm rate* (fraction where B strictly lowers nDCG@12) — the last being the operationally decisive number for an always-on deployment. Analysis: per-stratum Wilcoxon signed-rank on paired differences with Holm correction across strata; effect sizes with bootstrap confidence intervals. Sample-size honesty: with realistic labeling budgets (30–50 queries per stratum), the study is powered to detect medium effects only; smaller effects will be reported as estimation with intervals rather than tests.

5.5 Implementation

The protocol is implemented as four standard-library Python programs that pool candidates, emit a blinded labeling interface, ingest grades, and compute the analysis above. Three implementation choices affect what the resulting numbers mean. (i) Pooling invokes the deployed retrieval program rather than reimplementing it, and imports the deployed entity-gate predicate rather than restating it: a harness carrying its own copy of the system measures the copy, and in this deployment a duplicated batch pipeline had previously drifted into a silent index/model mismatch and produced no usable rows for a month. (ii) Condition provenance — which system retrieved a document, at which rank, and whether it arrived through the graph — is held in the database and never reaches the labeling interface, which receives only title, date, snippet, and an opaque key, in an order shuffled by a per-query deterministic seed. (iii) Queries whose pool is incompletely graded, whose pool contains no relevant document, or for which either condition returned nothing are excluded from aggregation and reported as counts; the last case matters because an empty ranking scores nDCG 0 and would otherwise enter the harm rate as a maximal loss caused by a failed run rather than by the graph.

One operational figure bounds study size: a single query costs approximately four minutes of wall-clock time, dominated by model loading in a fresh process, so a design of 30–50 queries in each of five strata is an overnight batch rather than an interactive session.

5.6 Replicability without data disclosure

The corpus is private and cannot be released. The protocol is therefore designed to be *portable rather than reproducible on our data*: the retrieval layer is public (MIT), the stratification rules, labeling instructions, ablation grid, and analysis code ship with it, and any owner of a linked Markdown corpus can execute the identical protocol on their own vault. We view a small number of independent single-corpus replications as the appropriate evidence standard for personal-corpus retrieval, where no shareable benchmark corpus can exist by definition.

6 Threats to validity

- **Single subject, single corpus.** All pilot data and the initial protocol execution concern one user’s corpus with one curation style; wikilink density and hub structure vary across users, and effects may not transfer.
- **Owner-as-labeler bias.** Relevance labels are produced by the corpus owner, who also built the system. Blinding of condition and randomization of output order mitigate but do not eliminate this bias.
- **Small pilot.** The $N=10$ telemetry is descriptive; no inferential claim is made or should be inferred from it.
- **Query-log conditioning.** Sourcing queries from production logs reflects real usage but under-samples query classes the user learned not to ask because the system handled them poorly (survivorship in the log).
- **Pool-relative recall.** Documents that neither condition retrieved are never labeled, so recall@12 is measured against the union of the two systems’ outputs rather than against the corpus. This is standard pooling practice and it flatters both conditions equally, but it is not corpus recall and should not be read as such.
- **Stratum construct validity.** The five strata are hypothesis-driven, not exhaustive; misclassification is measured via annotator disagreement but residual ambiguity (e.g., entity-flavored theme queries) remains.
- **Pre-rerank measurements.** Some motivating telemetry (index-composition statistics) is dense-retrieval-only and directional.

7 Conclusion

We described a deliberately minimal production Graph RAG system whose graph is human-curated wikilinks consumed at query time, reported small- N pilot telemetry indicating that the effect of graph expansion is conditioned on query class, and specified a stratified, paired, ablation-equipped evaluation protocol designed to answer *for which query classes* graph expansion helps retrieval over personal corpora. The protocol and code are public; execution of the full protocol on the deployment corpus, and invitation of independent replications on other private corpora, are the immediate next steps.

References

- [1] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, J. Larson, “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” arXiv:2404.16130, 2024.
- [2] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, D. Chalef, “Zep: A Temporal Knowledge Graph Architecture for Agent Memory,” arXiv:2501.13956, 2025.
- [3] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, “HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models,” *NeurIPS*, 2024. arXiv:2405.14831.
- [4] W. Xu et al., “A-MEM: Agentic Memory for LLM Agents,” arXiv:2502.12110, 2025.

- [5] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.
- [6] R. Nogueira, K. Cho, “Passage Re-ranking with BERT,” arXiv:1901.04085, 2019.
- [7] E. M. Voorhees, D. K. Harman (eds.), *TREC: Experiment and Evaluation in Information Retrieval*, MIT Press, 2005.